Week 8: Debugging with AI

AI Development Course

Midterm check-in: the peer demo

Tested with Superpowers 6.4.1 (its systematic-debugging skill), Claude Code 2.1.296 and the deal screener's real bug, fixed test first: Python 3.14.4, and Docker Engine 29.8.0 with python:3.12-slim (Python 3.12.15), 12 October 2026. The peer demo, everything inside a signed-in session, everything on GitHub, and Mac and Windows laptops are being tested.

Today

- The "fix it" loop, and the four steps instead
- Midterm peer demo: show version 1, try a classmate's, fill in the form
- Log the bugs in `BUGS.md`, as exact steps anyone can repeat
- Fix one bug with `systematic-debugging`: cause first, failing test, then one fix
- What to do after two failed attempts
- Push, read Files changed, merge
- The assignment, and next week

The "fix it" loop vs the four steps

The "fix it" loop	The four steps
"It's broken, fix it."	1. **Reproduce:** make it happen on purpose, with the exact input
Claude guesses a cause and changes code	2. **Narrow down:** find where and why, before changing anything
Something else breaks. "Still broken, fix it."	3. **Test:** a test that fails because of the bug, seen failing
Another guess, another change... five half-fixes, no known cause	4. **Fix:** one change, then every test passes

Slow at the start, fast at the end.

Superpowers' `systematic-debugging`

Installed with Superpowers in week 5. Its rule:

```
NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST
```

Course step	Skill's phase
Reproduce	1. Root Cause Investigation: read the error, "Reproduce Consistently"
Narrow down	1, then 2. Pattern Analysis (compare with what works) and 3. Hypothesis and Testing (one guess)
Test	4. Implementation, step 1: "Create Failing Test Case", "MUST have before fixing"
Fix	4, steps 2 and 3: "ONE change at a time", then verify

Quotes from skills/systematic-debugging/SKILL.md, Superpowers 6.4.1. It may start on its own when you describe a bug (being tested); this week you start it by name.

Midterm peer demo (in class)

Pairs, about **5 minutes each**, then swap.

Before: run your program once. **Hide anything private:** close `.env`, files with keys or personal data, Claude Code (`/exit`), your browser. Only the program's window shows.

Your demo: one sentence on what it does, one normal example, then **hand over the keyboard**. Watch, don't defend. Write down surprises.

Their demo: you try it and fill in `feedback-form.md`: exact inputs, what worked, what broke, one suggestion.

Printed, or copied into your repo as feedback-for-<partner>.md. You hand in the one you fill in for your partner (the file or a photo). Give the owner a copy. Being tested.

The odd-input list

Try each one where the program asks for something:

- blank (press Enter), words instead of numbers (`abc`)
- `0` , a negative number, a very large number
- a decimal where a whole number is expected (`30.5` years), and the reverse (`0.07` where `7` is meant)
- symbols: `$` , `%` , `,`
- very long text, and pressing Enter too early

"What broke" needs three things: **what I typed**, **what I expected**, **what happened** (messages word for word). "It crashed" isn't enough.

If the demo can't go as planned

- **Your build is broken today:** demo what runs, show the break. The break is your first bug: write down exactly what you typed and what it printed
- **Partner absent, or an odd number:** tell your instructor. Join another pair, or demo to the instructor
- **Partner found nothing:** go through the odd-input list together, item by item. A form that lists everything that worked is still a good form. Then try the list yourself, on parts they didn't reach

A confusing message, an error screen or a wrong number all count as bugs.

Branch first, and what to answer

`cd ~/semester-project && claude`, `manual mode on`, clean `git status`, then "Switch to main and pull ... make a new branch called **fix-<short-name>** and switch to it." (for example `fix-rate-decimal`)

Claude asks to	Choose
Create or edit `BUGS.md`, add a test for the bug	**Yes**
Edit the code where the cause is, **after** it explained the cause and you saw the test fail	**Yes**
`git add` / `commit` / `push` on `fix-...`, `gh pr create`	plain **Yes**
Edit anything unrelated to the bug ("while I'm here")	**No**
Delete or skip a test, or change its expected answer to pass	**No**
Any install	**No**
Change `main`, `reset --hard`, `--force`, `git clean`, `rm -rf`	**No**

Log the bugs: the `BUGS.md` template

Reproduce each bug yourself first (second terminal window, three fresh runs). Then one section per bug:

```
## Bug N: <short name>

- **Found by:** <who> on <date>
- **What I did (exact input):**
- **What I expected:**
- **What happened:**
- **Does it happen every time?** Yes / No / Not sure
- **Status:** Open / Fixed in <commit or PR link>
- **Cause:** (fill in once you know)
```

Word for word from example-project/BUGS.md. The README has the prompt that makes your BUGS.md from your form. Commit: "Log bugs from the peer demo".

The deal screener's Bug 1

```
## Bug 1: Interest rate typed as 0.07 is read as 0.07%

- **Found by:** the instructor, acting as a classmate, on 2026-10-12, using the
  odd-input list from the feedback form
- **What I did (exact input):** listing B (price 150000, rent 1700, tax 2400,
  insurance 1200, HOA 0, down payment 25, loan term 30, closing costs 6000), with
  `0.07` typed for the interest rate instead of `7`
- **What I expected:** a monthly mortgage payment of about $748.47, the 7% figure
  worked out by hand, or a question asking whether I meant 7%
- **What happened:** no question. It took the rate as 0.07%: monthly mortgage payment
  $315.80 (not $748.47), monthly cash flow $863.20 (not $430.53), cash-on-cash
  return 23.81% (not 11.88%). The verdict was Overall: PASS, the same as with `7`,
  so only the numbers show the mistake.
- **Does it happen every time?** Yes
- **Status:** Open
- **Cause:** (fill in once you know)
```

$315.80 vs $748.47 a month. Both runs said Overall: PASS: only the numbers show the bug.

Start `systematic-debugging`

In Claude Code, then paste your bug's whole section from `BUGS.md` after it:

```
/superpowers:systematic-debugging Debug the bug from BUGS.md in my project
that I paste below. First reproduce it and find the cause, and explain the
cause to me in plain English before you change anything. Then add a test to
my project's existing tests that fails because of this bug, using the exact
input from BUGS.md and the answer I expected, run it, and show me it failing.
Then stop. Don't change the code or fix anything yet. Don't change anything
unrelated to this bug, and never delete or weaken a test. Here is the bug:
```

- The command goes at the **very start** of the message (being tested)
- Not in the list? `/reload-plugins`, then try again

What you'll see, phase by phase (being tested)

1. **Root Cause Investigation.** *Deal screener:* no error, so the clue was the numbers. `parse_answer` accepts any percentage from 0 to 100, so `0.07` stayed 0.07%. The calculator was right
2. **Pattern Analysis.** *Deal screener:* `7` gives $748.47; the only difference is the typed rate. The down payment question had the same problem
3. **Hypothesis and Testing.** One guess, written down: "nothing asks whether a decimal like 0.07 meant 7%"
4. **Implementation, step 1:** a test that fails because of the bug. Then it **stops**, as you asked

Changing code before explaining the cause? **No**: "Explain the cause in plain English first."

The test fails first, then passes

Second terminal window, `python3 -m unittest`. **Before the fix** (shortened):

```
....................................F.FF.FF.
FAIL: test_rate_typed_as_a_fraction_asks_and_yes_means_percent (tests.test_main.SmallPercentTest...)
AssertionError: 'You typed 0.07. Did you mean 7%? Type y for 7%, or n to keep 0.07%: '
not found in [..., 'Interest rate (%): ', 'Loan term (years): ', 'Loan term (years): ', ...]
FAIL: test_yes_and_no_ignore_case_and_spaces (tests.test_main.SmallPercentTest...)
First differing element 0:
'Monthly mortgage payment: $315.80'
'Monthly mortgage payment: $748.47'
Ran 47 tests in 0.004s

FAILED (failures=5)
```

After the fix:

```
Ran 47 tests in 0.004s

OK
```

Screenshot both. The failure must show **your bug** (no question asked; the wrong answer), not `ImportError` or `NameError`. 5 of the 8 new tests failed; 3 check answers that already worked.

One fix, then check everywhere

Commit the test first: "Test for bug N: ... (fails before the fix)". (The deal screener used one commit; yours shows the test failing first.) Then:

```
Now make one fix: the smallest change to the code that fixes the cause you
found. Don't change the test, and don't change anything unrelated to this
bug. Then run all my tests and show me the result.
```

Deal screener: one change. It now asks:

`You typed 0.07. Did you mean 7%? Type y for 7%, or n to keep 0.07%:`

Check: **all** tests pass, the bug's exact input works, then in Docker (Docker Desktop running): `docker build -t semester-project .` and your test command in the container. Commit: "Fix bug N: ...".

When you're in the loop: two attempts, then stop

1. `Esc`. **Undo** with week 3's safe undo: `git restore` **all** uncommitted changes (only failed attempts are uncommitted), `git revert` committed attempts. The committed test stays
2. **Ask for the cause** before any change:

```
Before you change anything else, explain in plain English what you now think
causes this bug, what evidence shows it, what the two attempts changed, and
why they didn't work. Then tell me the one change you would make next, and
why it fixes the cause rather than hiding it. Don't make it yet.
```

The skill's own limit is **three**: "DON'T attempt Fix #4 without architectural discussion". The course stops at **two**: your attempts cost time and usage. Third one fails? Office hours.

"Discuss with your human partner before attempting more fixes": that's you. /rewind doesn't undo commands' changes, so use git.

Finish

1. "Push the fix-<short-name> branch to GitHub and open a pull request into main titled 'Fix bug N: ...'."
2. `BUGS.md`: Status "Fixed in <the pull request's address>", Cause in one or two plain sentences. Commit and push again: the PR updates
3. **Files changed:** the test, the fix, `BUGS.md` (and your feedback file). **Nothing else:** no deleted test, no changed expected answer, no `.env`
4. **Merge pull request**, then "Switch back to main and pull the latest changes from GitHub."

Committed to main? Week 5's rescue steps, with rescue-fix-<short-name>.

When it goes wrong

What you see	What to do
You can't make the bug happen	Ask exactly what they typed; fresh start; same version? Still no: `Not sure`, fix another bug
It happens only sometimes	Note what differs each time; ask Claude to compare, changing nothing
Claude wants to rewrite everything	**No**: "Only change what this bug needs"
The new test passes before the fix	It doesn't catch the bug: use the exact input and expected answer
Claude changed or deleted a test	**No**; already done? `git restore` or `git revert`
Tests pass on your laptop, not in Docker	Build the image again first; then ask why, changing nothing
It committed to `main`	Week 5's rescue

Assignment

- The **feedback form** you filled in for your partner (file or photo)
- The link to **`BUGS.md`** on `main` after the merge (not merged? on your fix branch): at least one bug with exact steps to reproduce
- The **merged fix PR**, with screenshots: the test **failing** before, all tests **passing** after
- Reflection: what your partner found that you hadn't; how you avoided (or got out of) the "fix it" loop

Criterion	Points
Feedback form: specific inputs and what happened	2
`BUGS.md`: at least one bug with exact reproduce steps	2
Merged fix PR: test failed before, passes after, nothing unrelated	4
Reflection	2

Next week: Calling a model from code.